

Hauptklasse:

```
public class Hauptklasse extends JFrame {

    private boolean packFrame = true;           //Attribute
    private Model model = new Model();          //Model Erzeugen
    private Controller controller = new Controller(model); //Controller Erz.
    private PanelNord pnNord = new PanelNord(controller); // Panels Erzeugen
    private PanelZentrum pnZentrum = new PanelZentrum(controller);
    private PanelSued pnSued = new PanelSued(controller);

    public GuiMvcObserverDrill() {

        model.addObserver(pnNord);              // Verbindung Model mit Observer
        model.addObserver(pnZentrum);           // mit allen View Elementen
        model.addObserver(pnSued);

        add(pnNord, BorderLayout.NORTH);        // Panels hinzufügen und
        add(pnZentrum, BorderLayout.CENTER);     plazieren
        add(pnSued, BorderLayout.SOUTH);

        model.notifyObservers();                // Model und Observer verbinden
    }
}
```

View

```
public class PanelNord extends JPanel implements ItemListener, Observer {
    //Panel Klasse

    public Controller controller;                //Controller aufrufen
    private CheckboxGroup group1 = new CheckboxGroup(); // Checkboxes erstellen
    private Checkbox checkbox0;
    private Checkbox checkbox1;
    private static JTextArea txtArea = new JTextArea(10, 40); // Textfelder

    public PanelNord(Controller controller) {    //Konstruktor
        super(new GridLayout(1, 4));           //Layout definieren
                                                // (1zeiler, 4spalten)
        super(new BorderLayout());              // oder BorderLayout
        this.controller = controller;          //Controller erzeugen
        setPreferredSize(new Dimension(600, 50)); // Dimension
        setBorder(MyBorderFactory.createMyBorder("PanelNord"));
                                                // Panel nach Factory gestalten

        checkbox0 = new Checkbox("FA18", group1, true); //Checkboxes
                                                        erzeugen
        checkbox1 = new Checkbox("Porsche", group1, false);
        JLabel label0 = new JLabel();
        JLabel label1 = new JLabel();
        add(checkbox0);
        add(label0);                             //Label erzeugen
        add(label1);
        add(checkbox1);
        checkbox0.addItemListener(this);
        checkbox1.addItemListener(this);
    }
}
```

```
add(txtArea, BorderLayout.CENTER);
add(btClear, BorderLayout.SOUTH);

setBackground(new Color(200, 200, 200)); //Hintergrund Farbe

public void itemStateChanged(ItemEvent e) { // Checkboxes auslesen

    InfoText.info("->itemStateChanged");

    if (e.getSource() == checkbox0) {
        controller.setIndex(0);
    }
    if (e.getSource() == checkbox1) {
        controller.setIndex(1);
    }
}

public void update(Observable observable, Object object) { // Kontakt mit
    InfoText.info("->PanelZentrum: update()"); // Model. Muss nicht
    Model model = (Model) observable; // jede View Klasse
    bild = model.getBild(); // Haben. Nur mit
    repaint(); // Rückgaben
}

public void paintComponent(Graphics g) {

    super.paintComponent(g); // paint von
    InfoText.info("->PanelZentrum: paintComponent()"); // Superklasse
    g.drawImage(bild, 20, 20, this);
}
```

Wenn Panel in Panel aufgerufen werden muss → siehe Fourier klasse PNCenter

Controller

```
public class Controller {
    private Model model;

    Controller(Model model) {
        this.model = model;
    }

    public void setIndex(int index) {
        InfoText.info("->Controller:setIndex");
        model.setIndex(index);
    }
}
```

Model

```
public class Model extends Observable {

    private int index;
    private Image[] bilder = new Image[2];           // Bilder Array für 2 Bilder

    public Model() {                                // Laden der Bilder aus
        for (int i = 0; i < bilder.length; i++) {    Array mit for schlaufe
            bilder[i] = Utility.LoadResourceImage("bild" + i + ".jpg");
        }
    }

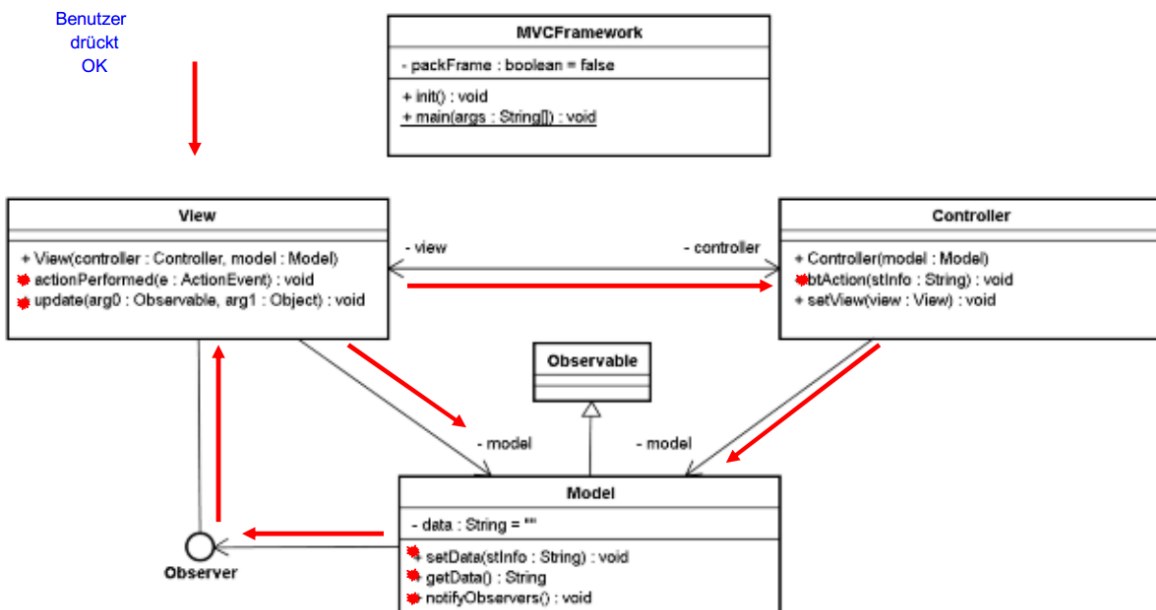
    public Image getBild() {                         // Alle Methoden die
        InfoText.info("->Model: getBild()");         Rechen oder etwas
        return bilder[index];                       Aufrufen müssen
    }

    public int getIndex() {
        return index;
        berechne Fourier();                         // Methoden aus eigener
                                                    Klasse aufrufen
    }

    public void setIndex(int index) {
        InfoText.info("->Model: SetIndex");

        this.index = index;
        notifyObservers();                           //Notifizieren Observer
    }

    public void notifyObservers() {                  // Verbindung zu Observer
        setChanged();
        super.notifyObservers();
    }
}
```



Button befindet sich auf Panel jp

```
jbFadeIn.addActionListener(this);  
jp.add(jbFadeIn)
```

Baut das ProcessingPane mit dem EditPanel auf dem Tab "Edit"

```
editPanel= new EditPanel(controller);  
add("Edit", editPanel);
```

Ruft die entsprechende Methode des Controller auf (ActionPerformed Methode)

```
if (e.getSource() == jbFadeIn) {  
    controller.fadeIn();  
}
```

Ruft update() von SignalPanel auf.

```
public void update(Observable obs, Object arg) {  
    // 1  
    signalPanel.update(obs, arg);  
}
```

Setzt den Preferred-Size des SignalPanel auf 500X400

```
signalPanel.setPreferredSize(new Dimension(500,400));
```

Das SignalPanel soll beim Vergrößern den gesamten Zuwachs kriegen.

```
setLeftComponent(signalPanel);
```

oder nicht

|| !

und

&&

Layout definieren

```
super(new GridBagLayout());
```

nicht editierbares Tf

```
tfVolt.setEditable(false);
```

Menübar

```
public class MeineMenuBar extends JMenuBar implements ActionListener{
    private JMenu menuDatei = new JMenu("Datei");
    private JMenuItem ladenItem = new JMenuItem("Datei öffnen");
    private JMenuItem speichernItem = new JMenuItem("Datei speichern");
    private JMenuItem exitItem = new JMenuItem("exit");

    public MeineMenuBar(){
        ladenItem.addActionListener(this);
        menuDatei.add(ladenItem);
        menuDatei.add(speichernItem);
        speichernItem.addActionListener(this);
        menuDatei.addSeparator();
        menuDatei.add(exitItem);
        exitItem.addActionListener(this);
        add(menuDatei);
        menuDatei.addSeparator();
    }

    public void actionPerformed(ActionEvent e) {
        if(e.getSource()==exitItem){
            System.exit(0);
        }
        System.out.println("+e.getSource()");

        if (returnVal == JFileChooser.APPROVE_OPTION) {
            File file = fc.getSelectedFile();
            System.out.println(file.getPath());
            controller.loadSignal(file.getPath());
        } else {
            System.out.println("Open command cancelled by user.");
        }
    }
}
```

GridBagLayout

1)erste überdeckte Zelle von links her;2) die erste überdeckte Zelle von oben her;
3)wieviele Zellen in horizontaler Richtung überdeckt werden; 4)wieviele Zellen in
vertikaler Richtung überdeckt werden;5)wachsen in x Richtung;6) wachsen in y-Richtung

```
add(label, new GridBagConstraints(5, 0, 1, 1, 0.0, 0.0, GridBagConstraints.SOUTH,
    GridBagConstraints.NONE, new Insets(5, 5, 5, 5), 0, 0));
```

konvertiert modelObject in ein Model

```
Model model=(Model) modelObject;
```